Runtime Evaluation. Table 1 presents average runtime for the `Naive` and `Optimized` calibration procedures across varying prompt budgets. While the `Naive` method can have a long runtime due to its unbounded censorship times C_i, it is observed to be faster than the `Optimized` method; see Table 1. We explain this by two key factors. First, our implementation of the `Naive` method includes an efficient runtime optimization: instead of always generating all C_i samples, we terminate generation early when additional samples no longer improve the estimate of $\hat{\alpha}(\tau)$. As detailed in Section E.3.1, this strategy yields LPBs that are equivalent to those produced by the original `Naive` procedure, while significantly reducing computation time. Second, the base model used in these experiments tends to produce low predicted quantiles $\hat{q}_\tau(X_i)$, as shown in Figure 2. Consequently, the effective censorship budgets C_i are also low for most prompts, further limiting runtime.

In sum, these two factors—runtime-efficient implementation and tendency to produce low predicted quantiles—explain why the reported runtime of the `Naive` is lower than the `Optimized` method in this setting.

Budget per prompt	Naive runtime (hours)	Optimized runtime (hours)	# GPUs
10	0.032 ± 0.005	0.063 ± 0.001	6
25	0.042 ± 0.001	0.117 ± 0.001	6
50	0.070 ± 0.006	0.209 ± 0.001	6
100	0.124 ± 0.003	0.388 ± 0.003	6
200	0.215 ± 0.010	0.589 ± 0.007	6
300	0.316 ± 0.012	0.718 ± 0.002	6
600	0.561 ± 0.016	0.979 ± 0.006	6
1200	0.918 ± 0.051	1.792 ± 0.041	4

Table 1: Average runtime (hours) by method and budget per prompt (mean ± SD).

Model Architecture and Training. We employ the ModernBERT-base (Warner et al., 2024) model. We use Adam optimizer (Kingma and Ba, 2015) to minimize the loss function in (35), with a learning rate of 3e-5, and a batch size of 1500. We train the model for 10 epochs and use early stopping based on the validation set.

E.3 Shared Implementation Details for Both Real and Synthetic Experiments

Our calibration framework applies to any machine learning model that outputs an estimated conditional quantile of the time-to-unsafe-sampling, denoted by $\hat{q}_\tau(x)$, given the covariates x. This is motivated by the fact that T, the time-to-unsafe-sampling, is a Geometric random variable with parameter $p(x)$, the probability of unsafe generation. Consequently, the conditional quantile function of T is given analytically in terms of $p(x)$, and vice versa:

$$q_\tau(x) = \left\lceil \frac{\log(1-\tau)}{\log(1-p(x))} \right\rceil \quad \text{and} \quad p(x) = 1 - (1-\tau)^{1/q_\tau(x)}.$$

The above relationship allows us to estimate either $p(x)$ or $q_\tau(x)$ and recover the other via a closed-form transformation.

Loss Function. In our experiments, we estimate $p(x)$ by fitting a model that minimizes the binary cross-entropy (BCE) loss over aggregate unsafe proportions. Specifically, for each prompt X_i, we define the empirical success rate as $\bar{Y}_i = \frac{1}{N}\sum_{j=1}^{N} Y_i^j$, where $Y_i^j \in \{0,1\}$ indicates whether the j-th sample is unsafe. We then use the BCE loss:

$$\mathrm{BCE}(\bar{Y}_i, \hat{p}(X_i)) = -\big[\bar{Y}_i \log \hat{p}(X_i) + (1-\bar{Y}_i)\log(1-\hat{p}(X_i))\big], \tag{35}$$

where $\hat{p}(X_i)$ is the model's prediction of the probability of unsafe generation. Notably, this loss is equivalent to the mean of the standard BCE loss across individual samples:

$$\frac{1}{N}\sum_{j=1}^{N} \mathrm{BCE}(Y_i^j, \hat{p}(X_i)) = \mathrm{BCE}(\bar{Y}_i, \hat{p}(X_i)).$$

This aggregation allows for a more computationally and memory-efficient implementation compared to the use of the vanilla BCE.

E.3.1 Implementation Details of the `Naive` Calibration Procedure

In theory, the `Naive` calibration (Algorithm 4) can assign an unbounded censorship time per prompt, which leads to arbitrarily long running times. To avoid this, we restrict our search for the threshold τ to the compact set $\mathcal{T} \cap \tau_{\text{prior}}$ (see Section 6.3). This restriction in search space does not violate the method's validity, as it may only make it more conservative, as discussed in Section 6.1. This section also shows that assigning a censorship time $C_i \geq \hat{q}_\tau(X_i)$ is equivalent to drawing exactly $\hat{q}_\tau(X_i)$ generations. Conversely, setting $C_i < \hat{q}_\tau(X_i)$ amounts to not drawing any samples. Based on this observation, we implement a run-time efficient version of the `Naive` method that draws censorship times according to

$$C_i := \text{Ber}(g(X_i)) \cdot \hat{q}_{\tau_{\text{prior}}}(X_i),$$

where $g(x) = \mathbb{P}(\text{Geom}(\min(|\mathcal{I}_2|/B, 1) \geq \hat{q}_{\tau_{\text{prior}}}(X_i))$. It is important to note at this point that the average number of generations per prompt reported in Figure 1 is the simulated number of samples that the equivalent, non-optimized `Naive` solution would use. This implementation also implies that the `Naive` method has a much smaller overall number of generations, leading to a faster runtime.

E.4 Machine's Spec

The computational infrastructure used in the experiments are:

- **CPU**: Intel(R) Xeon(R) CPU E5-2683 v4 @ 2.10GHz, Intel(R) Xeon(R) Gold 5318Y CPU @ 2.10GHz, Intel(R) Xeon(R) Gold 6336Y CPU @ 2.40GHz.
- **GPU**: NVIDIA A40, NVIDIA TITAN X (Pascal), NVIDIA 2080 TI, NVIDIA RTX 2060 SUPER.
- **OS**: Ubuntu 20.04.6.